Python Notes

for AI / ML Engineers

Practical. Concise. Production-focused.

1.1 Data Types & Variables

The 5 core types

- **int** — whole numbers: x = 42
- **float** — decimals: lr = 0.001
- **str** — text: name = 'bert'
- **bool** — True / False (subclass of int — True==1, False==0)
- **None** — absence of value, like null elsewhere

Type conversion — when you need to switch types

```
age = int("25")          # str  → int
price = float("9.99")   # str  → float
label = str(100)         # int  → str
flag = bool(0)           # 0, "", [], None  →  False  (everything else → True)
```

type() vs isinstance()

Use **type()** when you need the exact type. Use **isinstance()** when subclasses are OK (preferred in production code).

```
x = 3
type(x) == int          # True  — exact match
isinstance(x, (int, float))  # True  — accepts either type
```

Mutable vs Immutable — critical for AI pipelines

Immutable objects **cannot be changed after creation**. Python caches and shares them safely. Mutable objects can be edited in-place — watch out for accidental shared references.

```
# IMMUTABLE: int, float, str, tuple, bool
a = "hello"
b = a
b = b + "!"        # b is a NEW string; a is unchanged

# MUTABLE: list, dict, set
weights = [0.1, 0.2]
backup = weights    # NOT a copy — same object!
backup.append(0.3)
print(weights)      # [0.1, 0.2, 0.3]  ← surprise!

# Fix: make an explicit copy
backup = weights.copy()
```

■ **Note:** In AI pipelines, passing mutable objects (dicts, lists) between functions without copying is a common source of silent bugs. Always copy when you don't want shared state.

1.1 Strings

Slicing — the most used string trick

```
s = "transformer"
s[0]        # 't'
s[-1]       # 'r'
s[0:5]      # 'trans'
s[::2]      # every 2nd char  →  'rnfre'
s[::-1]     # reverse  →  'remrofsnart'
```

Key methods

```
text = "  Hello, World!  "

text.strip()                    # remove whitespace from both ends
text.lower() / text.upper()
text.replace("World", "Claude")
text.split(", ")                # ['  Hello', 'World!  ']
"-".join(["a", "b", "c"])       # 'a-b-c'
text.startswith("  Hello")      # True
text.find("World")              # index of first match, -1 if not found
```

f-strings — use these always

```
name = "GPT"
loss = 0.023456

print(f"Model: {name}")
print(f"Loss: {loss:.4f}")          # 4 decimal places → 0.0235
print(f"Epoch {3:02d}/10")          # zero-padded  → Epoch 03/10
print(f"Result: {2 ** 10 = }")      # debug shortcut → Result: 2 ** 10 = 1024
```

Multiline strings

```
prompt = """
You are a helpful assistant.
Answer only in JSON format.
"""
# Leading/trailing newlines are included — strip() if you don't want them
```

1.1 Collections

Lists — your everyday workhorse

```
tokens = ["hello", "world", "!"]

tokens[0]           # 'hello'
tokens[-1]          # '!'
tokens[1:]          # ['world', '!']

tokens.append("?")          # add to end
tokens.extend(["a", "b"])   # add multiple
tokens.pop()                # remove & return last item
tokens.pop(0)               # remove by index
tokens.sort()               # in-place sort (modifies list)
sorted(tokens)              # returns NEW sorted list
```

```
tokens.reverse()
len(tokens)
```

Tuples — immutable lists

Use tuples when the data should NOT change: coordinates, RGB values, function return pairs. They're slightly faster than lists.

```
point = (10, 20)
x, y = point           # tuple unpacking

# Common in AI: returning multiple values
def get_stats(data):
    return min(data), max(data), sum(data)/len(data)

lo, hi, avg = get_stats([1, 2, 3, 4, 5])
```

Dictionaries — key-value store, used everywhere in AI

```
config = {"lr": 0.001, "epochs": 10, "batch_size": 32}

# Read
config["lr"]                    # 0.001 (KeyError if missing)
config.get("dropout", 0.1)      # safe read with default

# Write / Update
config["epochs"] = 20
config.update({"optimizer": "adam", "lr": 0.0005})

# Delete
del config["batch_size"]

# Iterate
for key, value in config.items():
    print(f"{key}: {value}")

config.keys()    # dict_keys([...])
config.values()  # dict_values([...])
```

Sets — uniqueness + fast lookup

```
vocab = {"the", "cat", "sat", "the", "cat"}
print(vocab)        # {'the', 'cat', 'sat'}  — duplicates removed

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a | b   # union        → {1,2,3,4,5,6}
a & b   # intersection → {3,4}
a - b   # difference   → {1,2}
5 in b  # True — O(1) lookup, faster than list
```

Nested collections — real AI config patterns

```
# List of dicts — batch of training examples
batch = [
    {"input": "hello", "label": 1},
    {"input": "bye",   "label": 0},
```

```
]
for sample in batch:
    print(sample["input"], sample["label"])

# Dict of lists — metrics per epoch
history = {"loss": [0.9, 0.7, 0.5], "acc": [0.6, 0.75, 0.85]}
print(history["loss"][-1])   # last epoch loss → 0.5
```

1.1 Control Flow

if / elif / else

```
score = 0.87

if score > 0.9:
    print("Excellent")
elif score > 0.75:
    print("Good")
else:
    print("Needs improvement")

# One-liner (ternary)
label = "pass" if score > 0.5 else "fail"
```

for loops — iterate over anything

```
layers = ["embed", "attn", "ffn"]

for layer in layers:
    print(layer)

# With index
for i, layer in enumerate(layers):
    print(i, layer)     # 0 embed, 1 attn, 2 ffn

# Dict iteration
params = {"lr": 0.001, "epochs": 10}
for k, v in params.items():
    print(k, "=", v)

# Parallel iteration
names = ["Alice", "Bob"]
scores = [92, 87]
for name, score in zip(names, scores):
    print(name, score)

# Range
for i in range(0, 10, 2):   # 0,2,4,6,8
    pass
```

while, break, continue

```
epoch = 0
best_loss = float("inf")
```

```
while epoch < 100:
    loss = train_one_epoch()
    if loss < 0.01:
        print("Converged early!")
        break               # exit loop
    if loss > best_loss * 2:
        epoch += 1
        continue            # skip rest, go to next iteration
    best_loss = loss
    epoch += 1
```

1.2 Functions

Basics — def, return, docstrings

```
def tokenize(text, lower=True):
    """Split text into tokens. Lowercases by default."""
    if lower:
        text = text.lower()
    return text.split()

tokens = tokenize("Hello World")   # ['hello', 'world']
tokens = tokenize("Hello", lower=False)
```

*args and **kwargs — critical for AI SDK wrappers

***args** = variable positional args (tuple). ****kwargs** = variable keyword args (dict). You'll see these all over LangChain, HuggingFace, OpenAI SDK.

```
def log(*args):
    for item in args:
        print(item)

log("loss", 0.5, "acc", 0.9)   # prints each on its own line

# **kwargs lets you pass arbitrary keyword params
def call_model(prompt, **kwargs):
    # kwargs might be: temperature=0.7, max_tokens=200, top_p=0.9
    return api.complete(prompt, **kwargs)

call_model("Hello", temperature=0.7, max_tokens=100)

# Combining both — the most flexible signature
def wrapper(fn, *args, **kwargs):
    print(f"Calling {fn.__name__}")
    return fn(*args, **kwargs)
```

Tuple unpacking from return values

```
def train_epoch(model, data):
    # ... training code ...
    return 0.432, 0.891   # loss, accuracy

loss, acc = train_epoch(model, train_data)
```

```
# Ignore values you don't need
loss, _ = train_epoch(model, train_data)
```

Lambda — tiny throwaway functions

```
# Instead of defining a full function for simple transforms
double = lambda x: x * 2
double(5)   # 10

# Most common use: as key= in sort/filter
models = [{"name": "gpt4", "score": 0.91}, {"name": "llama", "score": 0.87}]
models.sort(key=lambda m: m["score"], reverse=True)
```

Recursion — brief

A function that calls itself. Classic use: tree traversal. Always needs a base case to stop. Python's default recursion limit is 1000. For AI work, you'll rarely write recursive code — but you'll read it in frameworks.

```
def factorial(n):
    if n <= 1: return 1        # base case
    return n * factorial(n-1)  # recursive call
```

1.3 Object-Oriented Programming

The essentials — class, __init__, self

Think of a class as a blueprint and an instance as the actual object. **self** is just the object itself passed automatically to every method.

```
class TextPipeline:
    default_lang = "en"         # class variable — shared by ALL instances

    def __init__(self, model_name, max_len=512):
        self.model_name = model_name   # instance variables — unique per object
        self.max_len = max_len
        self._cache = {}               # private by convention

    def process(self, text):
        text = text[:self.max_len]
        return text.lower().split()

    def __repr__(self):
        return f"TextPipeline(model={self.model_name})"

pipe = TextPipeline("bert-base", max_len=128)
pipe.process("Hello World")
print(pipe)   # TextPipeline(model=bert-base)
```

Inheritance — how LangChain / LlamaIndex work

You subclass a base class and override specific methods. This is the core pattern in every AI framework.

```
class BaseLLM:
    def predict(self, prompt):
        raise NotImplementedError
```

```
    def batch_predict(self, prompts):
        return [self.predict(p) for p in prompts]   # reuses predict()

class OpenAIWrapper(BaseLLM):
    def __init__(self, model="gpt-4o"):
        super().__init__()   # call parent __init__ if it has one
        self.model = model

    def predict(self, prompt):
        # actual API call here
        return call_openai(self.model, prompt)

llm = OpenAIWrapper()
llm.batch_predict(["Q1", "Q2"])   # inherited, calls our predict()
```

Abstract classes — enforced interfaces

Use ABC when you want to **force** subclasses to implement specific methods. LangChain's BaseChain, BaseRetriever etc. all use this pattern.

```
from abc import ABC, abstractmethod

class BaseEmbedder(ABC):
    @abstractmethod
    def embed(self, text: str) -> list[float]:
        pass   # subclass MUST implement this

class OpenAIEmbedder(BaseEmbedder):
    def embed(self, text):
        return openai.embed(text)   # actual impl

# BaseEmbedder()  ← would raise TypeError
# OpenAIEmbedder()  ← works fine
```

Useful dunder methods

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)        # enables: len(dataset)

    def __getitem__(self, idx):
        return self.data[idx]        # enables: dataset[0], dataset[1:5]

    def __repr__(self):
        return f"Dataset({len(self)} samples)"

ds = Dataset([1, 2, 3, 4])
len(ds)     # 4
ds[0]       # 1
ds[1:3]     # [2, 3]
```

@property, @staticmethod, @classmethod

```
class Model:
    def __init__(self, weights):
```

```
        self._weights = weights

    @property
    def size(self):
        return len(self._weights)    # access like attribute: model.size

    @staticmethod
    def validate_config(cfg):
        return "lr" in cfg           # no self/cls — utility function

    @classmethod
    def from_pretrained(cls, path):
        weights = load(path)
        return cls(weights)          # creates instance — alternative constructor

m = Model.from_pretrained("./bert")
m.size                               # no ()
Model.validate_config({"lr": 0.001})
```

1.4 Pythonic Code & Idioms

Comprehensions — replace for-loops that build collections

```
# List comprehension
tokens = ["hello", "world", "hi"]
upper = [t.upper() for t in tokens]
long = [t for t in tokens if len(t) > 3]   # with filter

# Dict comprehension
scores = {"alice": 92, "bob": 87, "charlie": 95}
passing = {k: v for k, v in scores.items() if v >= 90}

# Set comprehension
unique_lengths = {len(t) for t in tokens}

# Nested list comprehension (read inner first)
matrix = [[i * j for j in range(3)] for i in range(3)]
```

Generator expressions — memory efficient for large data

Generators produce values one-at-a-time instead of building the whole list in memory. Essential when processing large files or streaming model outputs.

```
# List — loads ALL into memory at once
squares_list = [x**2 for x in range(1_000_000)]   # ~8 MB

# Generator — computes on demand, tiny memory footprint
squares_gen = (x**2 for x in range(1_000_000))    # ~120 bytes

# Usage is the same
for val in squares_gen:
    process(val)

# sum/max/min work directly on generators
total = sum(x**2 for x in range(1000))
```

map(), filter(), reduce()

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

list(map(lambda x: x**2, nums))         # [1, 4, 9, 16, 25]
list(filter(lambda x: x % 2 == 0, nums))# [2, 4]
reduce(lambda a, b: a + b, nums)         # 15 (running total)

# In practice, comprehensions are more readable — use map/filter
# only when passing functions that already exist
list(map(str, nums))   # ['1','2','3','4','5']
```

Unpacking

```
a, b, c = [1, 2, 3]
first, *rest = [1, 2, 3, 4, 5]    # first=1, rest=[2,3,4,5]
*head, last = [1, 2, 3, 4, 5]     # head=[1,2,3,4], last=5

# Swap without temp variable
a, b = b, a

# Ignore values
_, important, _ = (10, 42, 99)
```

any(), all(), sorted() with key

```
scores = [0.8, 0.6, 0.9, 0.4]

any(s > 0.85 for s in scores)   # True  — at least one
all(s > 0.5 for s in scores)    # False — not all

# sorted with key
models = [{"name": "A", "f1": 0.91}, {"name": "B", "f1": 0.87}]
sorted(models, key=lambda m: m["f1"], reverse=True)
```

collections module — power tools

```
from collections import Counter, defaultdict, deque

# Counter — frequency count
words = ["the", "cat", "sat", "the", "the", "cat"]
c = Counter(words)
c.most_common(2)     # [('the', 3), ('cat', 2)]

# defaultdict — dict that auto-creates missing keys
dd = defaultdict(list)
for word, tag in [("run","VB"), ("fast","RB"), ("run","VB")]:
    dd[word].append(tag)
# {'run': ['VB','VB'], 'fast': ['RB']}

# deque — fast O(1) append/pop from both ends
q = deque(maxlen=3)   # sliding window
for item in [1,2,3,4,5]:
    q.append(item)
# deque([3,4,5])
```

1.5 File I/O & Data Handling

Reading / writing files — always use context managers

```
# Read
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()        # whole file as string
    lines = f.readlines()     # list of lines (includes \n)

# Lazy line-by-line (memory efficient for huge files)
with open("data.txt") as f:
    for line in f:
        process(line.strip())

# Write
with open("output.txt", "w") as f:
    f.write("hello\n")

# Append
with open("log.txt", "a") as f:
    f.write("new entry\n")
```

JSON — the AI world's native format

```
import json

# Load from file
with open("config.json") as f:
    config = json.load(f)

# Load from string (e.g., LLM response)
data = json.loads('{"key": "value"}')

# Save to file
with open("results.json", "w") as f:
    json.dump(results, f, indent=2)

# To string (for API responses)
json_str = json.dumps({"status": "ok"})
```

CSV

```
import csv

# Read
with open("data.csv") as f:
    reader = csv.DictReader(f)     # each row → dict
    for row in reader:
        print(row["name"], row["score"])

# Write
with open("out.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name","score"])
    writer.writeheader()
    writer.writerow({"name": "Alice", "score": 92})
```

Pickle — save Python objects directly

```
import pickle

# Save a trained model / tokenizer / any Python object
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

# Load it back
with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

■ **Note:** Never unpickle files from untrusted sources — it executes arbitrary code. Use JSON or safetensors for sharing models publicly.

pathlib — modern file paths

```
from pathlib import Path

data_dir = Path("./data")
data_dir.mkdir(parents=True, exist_ok=True)   # create if not exists

# Build paths cleanly (works on Windows + Linux)
model_path = data_dir / "models" / "bert.pkl"

model_path.exists()
model_path.suffix      # '.pkl'
model_path.stem        # 'bert'
model_path.parent      # Path('./data/models')

# List files
for f in data_dir.glob("*.json"):   # pattern matching
    print(f)
```

1.6 Error Handling & Debugging

try / except / finally

```
def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"Config not found: {path}")
        return {}
    except json.JSONDecodeError as e:
        print(f"Bad JSON: {e}")
        return {}
    finally:
        print("load_config done")   # always runs, even on error

# Catching multiple
except (ValueError, TypeError) as e:
    print(e)
```

```
# Catch all (use sparingly — hides bugs)
except Exception as e:
    print(f"Unexpected: {type(e).__name__}: {e}")
```

Raising exceptions

```
def set_lr(lr):
    if lr <= 0:
        raise ValueError(f"Learning rate must be positive, got {lr}")
    self.lr = lr

# Custom exception — good for library code
class ModelNotLoadedError(Exception):
    pass

def predict(text):
    if not self.model:
        raise ModelNotLoadedError("Call load_model() first")
```

Logging — better than print()

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

log = logging.getLogger(__name__)

log.debug("Token count: %d", len(tokens))   # only shown at DEBUG level
log.info("Starting training")
log.warning("Low memory: %d MB remaining", free_mb)
log.error("API call failed: %s", err)

# In production, set level to INFO or WARNING
# During dev, set to DEBUG for full detail
```

Debugging with breakpoint()

```
def process_batch(batch):
    result = []
    for item in batch:
        breakpoint()    # ← execution stops here, drops into pdb shell
        # pdb commands: n (next), s (step in), c (continue), q (quit)
        # p variable_name  to inspect any variable
        output = model(item)
        result.append(output)
    return result
```

1.7 Performance & Memory

Generators and yield — critical for streaming AI responses

A generator function uses **yield** instead of return. It pauses at each yield and resumes when the caller asks for the next value. This is exactly how streaming LLM responses work.

```
# Regular function — builds full list in memory
def get_tokens(text):
    return text.split()   # returns all at once

# Generator function — yields one token at a time
def stream_tokens(text):
    for token in text.split():
        yield token        # pauses here, sends token, resumes later

# Usage looks the same
for token in stream_tokens("hello world foo"):
    print(token)

# Real AI streaming pattern
def stream_llm_response(prompt):
    for chunk in openai_client.stream(prompt):
        yield chunk.text        # yield each chunk as it arrives

for piece in stream_llm_response("Tell me a story"):
    print(piece, end="", flush=True)
```

yield from — delegating to another generator

```
def process_documents(docs):
    for doc in docs:
        yield from tokenize(doc)   # yields every token from tokenize()
        # equivalent to: for t in tokenize(doc): yield t
```

itertools — lazy iteration utilities

```
import itertools

# chain — treat multiple iterables as one
for item in itertools.chain(list1, list2, list3):
    process(item)

# islice — take first N items from any iterable (even infinite)
first_100 = list(itertools.islice(huge_generator, 100))

# batched (Python 3.12+) — chunk into fixed-size batches
for batch in itertools.batched(dataset, 32):
    train(batch)

# product — cartesian product (great for hyperparameter search)
lrs = [0.01, 0.001]
batch_sizes = [16, 32]
for lr, bs in itertools.product(lrs, batch_sizes):
    run_experiment(lr, bs)
```

Benchmarking — timeit and cProfile

```
import timeit

# Quick timing
t = timeit.timeit("'-'.join(str(n) for n in range(100))", number=10000)
```

```
# Profile your whole script to find bottlenecks
import cProfile
cProfile.run("your_function()")

# Or from command line:
# python -m cProfile -s cumulative your_script.py
```

Shallow vs Deep copy

```
import copy

original = {"data": [1, 2, 3], "name": "train"}

# Shallow copy — top-level copied, nested objects still shared
shallow = original.copy()
shallow["data"].append(99)
print(original["data"])   # [1, 2, 3, 99]  ← mutated!

# Deep copy — fully independent clone
deep = copy.deepcopy(original)
deep["data"].append(99)
print(original["data"])   # [1, 2, 3]  ← safe
```

Vectorization — avoid Python loops on arrays

Pure Python loops are slow for numerical work. NumPy/PyTorch operations run in compiled C/CUDA — orders of magnitude faster. Always prefer vectorized ops over element-wise Python loops.

```
import numpy as np

data = np.array([1.0, 2.0, 3.0, 4.0])

# BAD — Python loop
result = []
for x in data:
    result.append(x * 2 + 1)

# GOOD — vectorized (runs in C, 100x+ faster for large arrays)
result = data * 2 + 1

# Applies to everything: sigmoid, softmax, normalization, etc.
sigmoid = 1 / (1 + np.exp(-data))
normalized = (data - data.mean()) / data.std()
```

■ **Note:** This is one of the most important habits for AI engineering. Benchmark first with cProfile, then vectorize the hot path.